Roller Coaster Physics

SNU Computer Science & Engineering

Roller Coaster

- Next programming assignment involves creating a 3D roller coaster animation



Roller Coaster Physics

Assumption

- a point mass moving on a spline
- frictionless with gravity
- energy conservation law

$$E_K + E_P = \frac{1}{2} m \|v\|^2 + mgh = mgh_{max}$$

- h_{max} reached when $\|v\| = 0$
- given h, we can always compute the velocity

$$\|v\| = \sqrt{2g(h_{max} - h)}$$

Motion on a Spline

Assumption

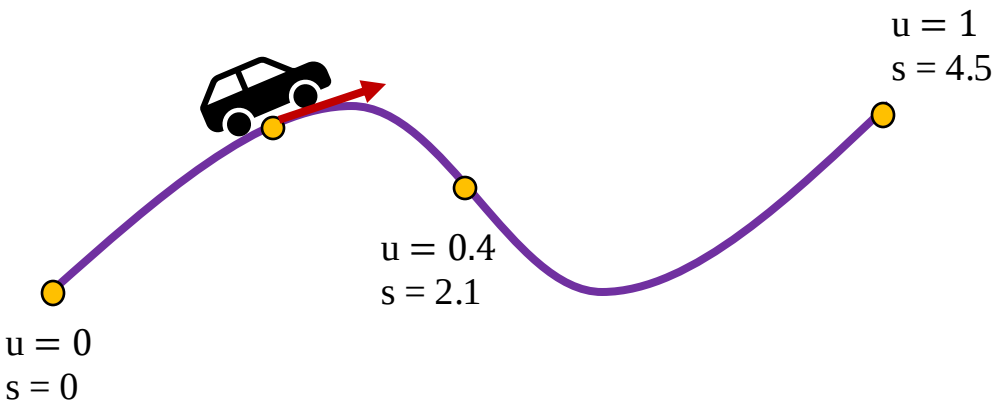
- the mass point moves on a parametric function $P(u) = (x(u), y(u), z(u))$
  - parameter u does not denote time
  - $u(t)$ is a function of time t
- we know the initial position of the point and its velocity $v = v(u)$ at any time u

Problem

- compute the position of the point at arbitrary time t

Arc-length Parameterization

- There are an infinite number of parameterizations of a given curve
- There is a special one: arc-length parameterization



- arc-length $s = s(u)$ is the distance from $P(0)$ to $P(1)$ along the curve
- parameterizations usually are not arc-length

Reparameterization

Length of a curve s(u)

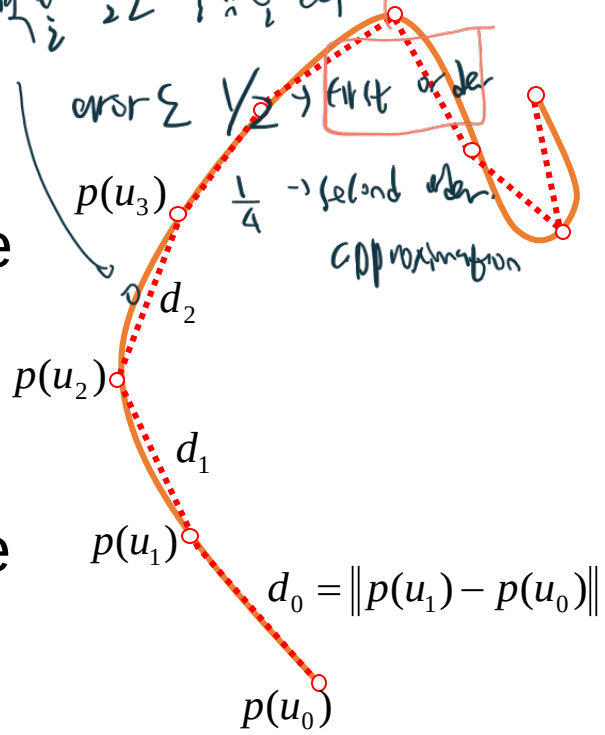
$$s = LENGTH(u, u_0)$$

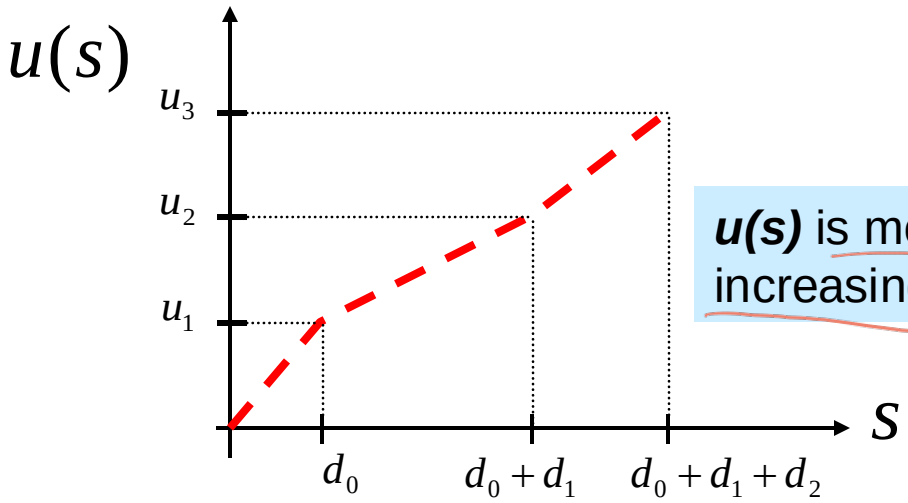
$$= \int_{u_0}^{u} \sqrt{\left(\frac{dx(\tilde{u})}{d\tilde{u}}\right)^2 + \left(\frac{dy(\tilde{u})}{d\tilde{u}}\right)^2 + \left(\frac{dz(\tilde{u})}{d\tilde{u}}\right)^2}\, d\tilde{u}$$

- $s(u)$ is monotonically increasing
- The integral cannot be evaluated analytically, not even for polynomials

Chord Length Approximation

- Sample the curve at a multitude of parametric values
  - Ex) $u_1, u_2, \cdots, u_n$
- Estimate the arc length by computing the linear distance through the sequence of samples
- We have to break the integral over individual polynomial pieces of the spline

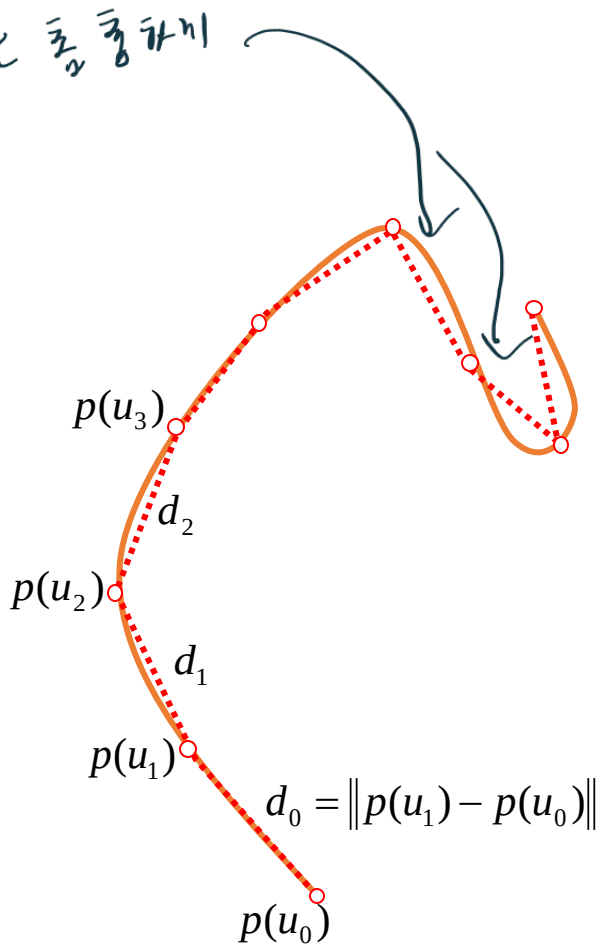




u(s) is monotonically increasing with respect to ***s***

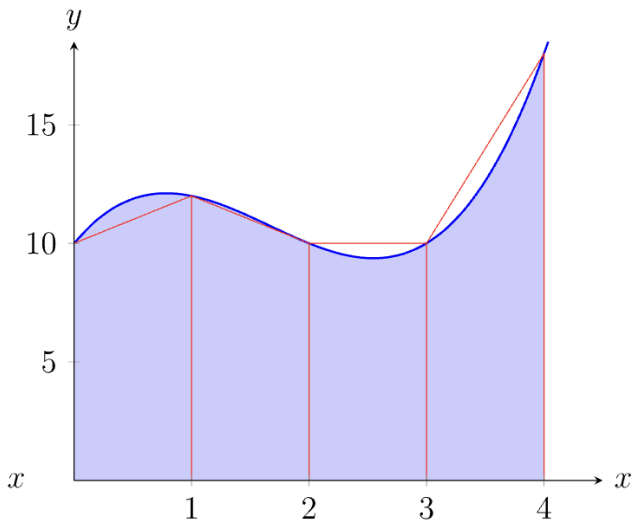
Chord Length Approximation

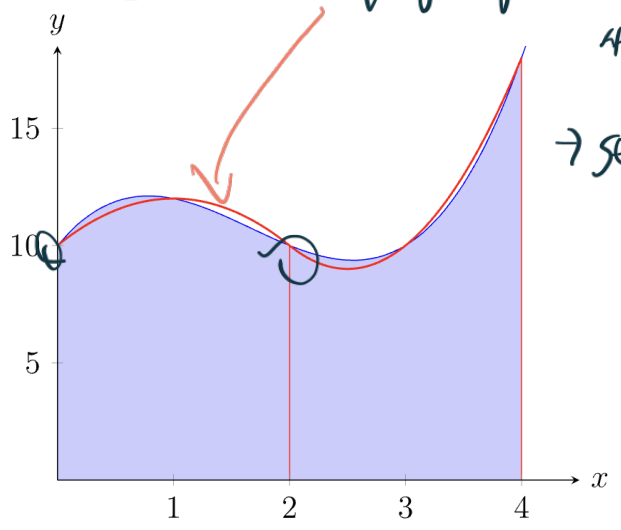
- Adaptive sampling
  - Add a new sample at the midpoint between two adjacent points ***p(u$_i$)*** and ***p(u$_{i+1}$)*** if the total length changes above given tolerance
  - Repeat until there is no more point to add



General Numerical Integration Methods

- Evenly spaced sample intervals
  - Trapezoidal rule (piecewise linear)
  - Simson's rule (piecewise quadratic)





https://www.sfu.ca/math-coursenotes/Math%20158%20Course%20Notes/sec_Numerical_Integration.html

- Gaussian quadrature rules are commonly used
- Adaptive sampling is also viable

Computing Inverse u=u(s)

- Given arc-length s, determine the original parameter u
- Since s(u) is monotonically increasing, so is u(s)
- can be formulated as a root finding problem

$$f(u) = s - LENGTH(u, u_0) = 0$$

- Newton-Raphson iteration is commonly used
- Alternatively, the bisection algorithm may also be applied
- The solution is unique if the function is strictly monotonic

Simulating mass point on a spline

- Assume we know the magnitude of the velocity vector $\|v(u)\|$ at any u
- Simulate the point with constant time step Δt
- Starting from $u = u_0, s = s_0, t = t_0$
- Update s and t:

$$s \Leftarrow s + \|v(u)\|\Delta t$$
$$t \Leftarrow t + \Delta t$$

- Update $u = u(s)$ by root finding

Camera movement along a Spline

Frenet frame

- well-defined and continuous as long as P' and P'' do not vanish



tangent vector

$$T(u) = \frac{P'(u)}{|P'(u)|}$$

binormal vector

$$B(u) = \frac{P'(u) \times P''(u)}{|P'(u) \times P''(u)|}$$

normal vector

$$\mathrm{N}(u) = B(u) \times T(u)$$

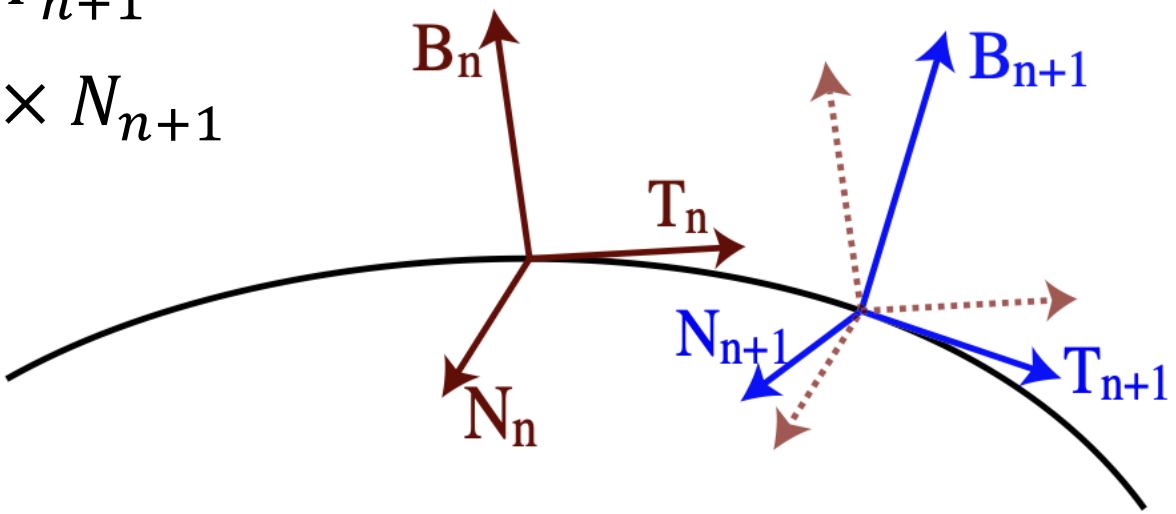
Camera movement along a Spline

Trick for constructing continuous frames

$$T_{n+1} = T(u_{n+1})$$

$$N_{n+1} = B_n \times T_{n+1}$$

$$B_{n+1} = T_{n+1} \times N_{n+1}$$



Not ideal in practice, why?

Unit Quaternion Splines

- a curve construction as affine combination of control points

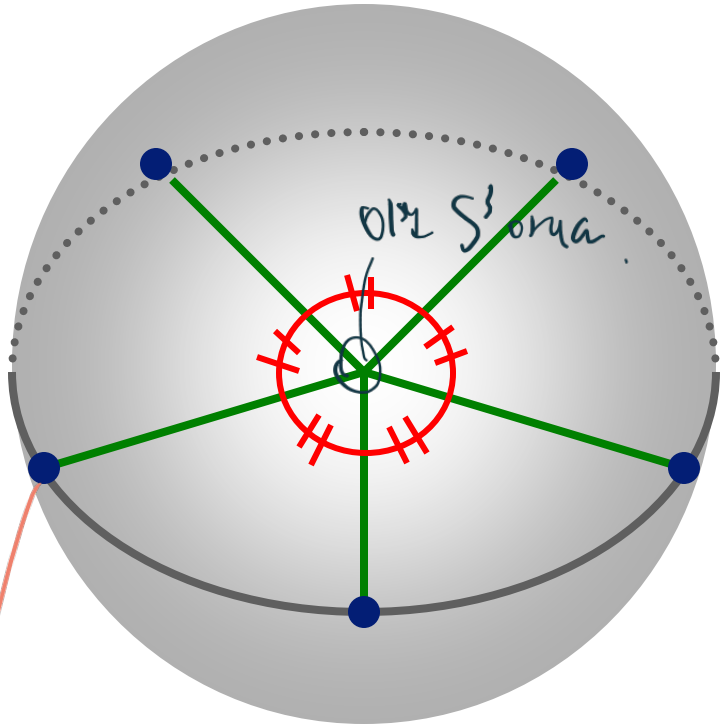
$$\mathrm{P}(t) = \boldsymbol{p}_0 B_0(t) + \boldsymbol{p}_1 B_1(t) + \boldsymbol{p}_2 B_2(t) + \boldsymbol{p}_3 B_3(t)$$

$$1 = \sum_{i=0}^{n} B_i(t) \quad \text{for any } 0 \leq t \leq 1$$

- we can construct a curve on unit quaternion space if an affine combination of unit quaternions is defined
- An affine combination in S^3 can be defined in various ways, leading to different outcomes
- Generalize slerp for n-points

Affine Combination in S^3

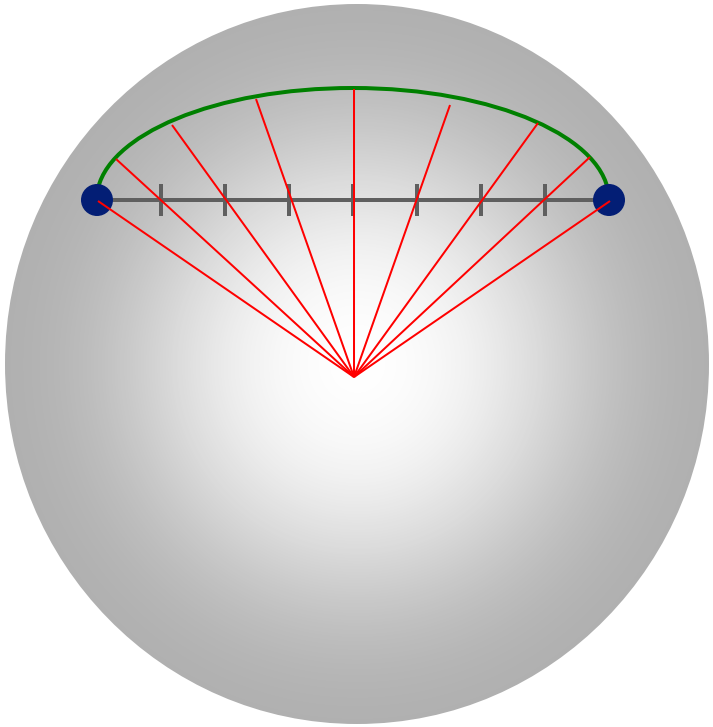
- Inherent ambiguity
  - Spherical structure
  - Antipodal equivalence
- Popular methods
  - Renormalization
  - Multi-linear
  - Linearization by exp/log
  - Functional optimization



Re-normalization

- Treat unit quaternions s 4D vectors when performing the affine combination, and
- subsequently normalize the result to project it back onto R^4

$$\boldsymbol{q} = \frac{w_0\boldsymbol{q}_0 + \cdots + w_n\boldsymbol{q}_n}{\|w_0\boldsymbol{q}_0 + \cdots + w_n\boldsymbol{q}_n\|}$$



- Pros
  - Simple, efficient
- Cons
  - Linear precision

Multi-Linear Method

- Evaluate n-point weight sum as a sequence of slerps

$$\mathbf{c} = \frac{1}{2}\mathbf{p}_1 + \frac{1}{4}\mathbf{p}_2 + \frac{1}{4}\mathbf{p}_3 \quad \Rightarrow \quad \mathbf{c} = \underbrace{\left(\frac{1}{2}\mathbf{p}_1 + \frac{1}{4}\mathbf{p}_2\right)}_{\text{Slerp}} + \underbrace{\frac{1}{4}\mathbf{p}_3}_{\text{Slerp}}$$

$$\mathbf{q}_c = slerp(1/4, slerp(1/3, \mathbf{q}_1, \mathbf{q}_2), \mathbf{q}_3)$$

Multi-Linear Method

$$\mathbf{c} = \frac{1}{2}\mathbf{p}_1 + \frac{1}{4}\mathbf{p}_2 + \frac{1}{4}\mathbf{p}_3$$

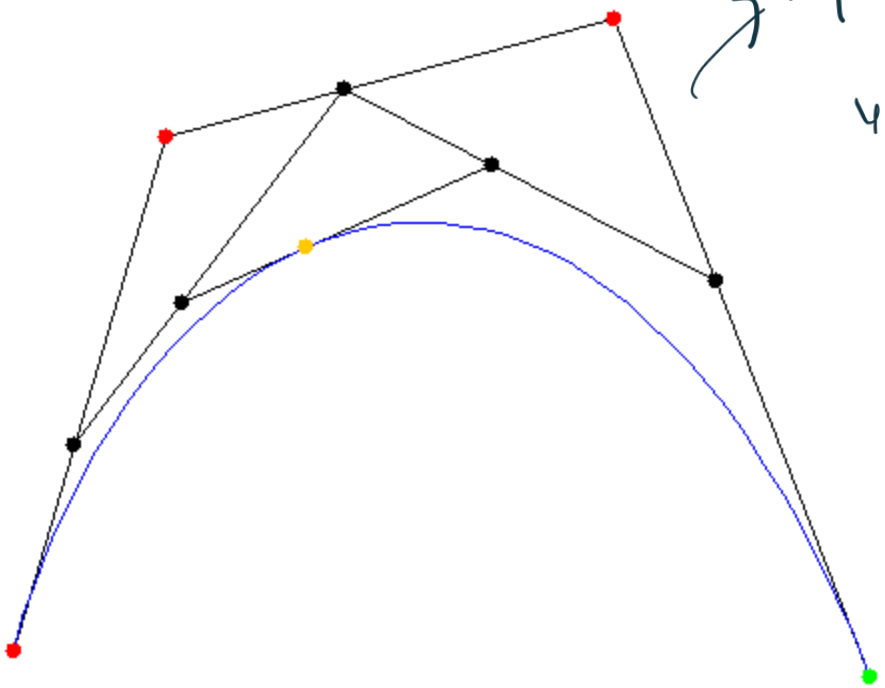
$$\mathbf{c} = \frac{1}{2}\mathbf{p}_1 + \left(\frac{1}{4}\mathbf{p}_2 + \frac{1}{4}\mathbf{p}_3\right) \qquad \mathbf{c} = \left(\frac{1}{2}\mathbf{p}_1 + \frac{1}{4}\mathbf{p}_2\right) + \frac{1}{4}\mathbf{p}_3$$

$$\mathbf{q}_c = slerp(\tfrac{1}{2}, \mathbf{q}_1, slerp(\tfrac{1}{2}, \mathbf{q}_2, \mathbf{q}_3)) \neq \mathbf{q}'_c = slerp(\tfrac{1}{4}, slerp(\tfrac{1}{3}, \mathbf{q}_1, \mathbf{q}_2), \mathbf{q}_3)$$

Slerp is not associative

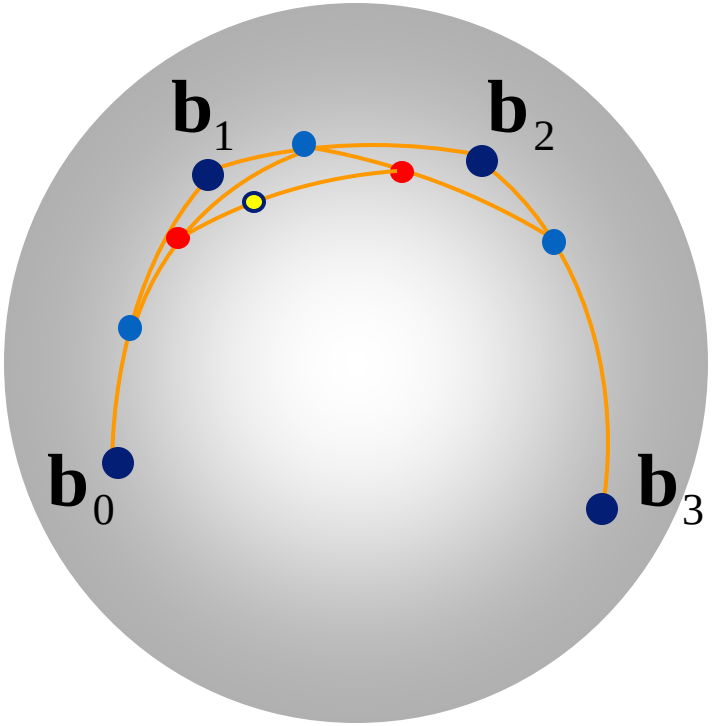
De Casteljau Algorithm

- A procedure for evaluating a point on a Bezier curve



Quaternion Bezier Curve

- Multi-linear construction
  - De Casteljau algorithm with Slerp



$$\mathbf{q}(0) = \mathbf{b}_0$$

$$\mathbf{q}(1) = \mathbf{b}_3$$

$$\omega(0) = \mathbf{q}^{-1}(0)\dot{\mathbf{q}}(0) = k\log(\mathbf{b}_0^{-1}\mathbf{b}_1)$$

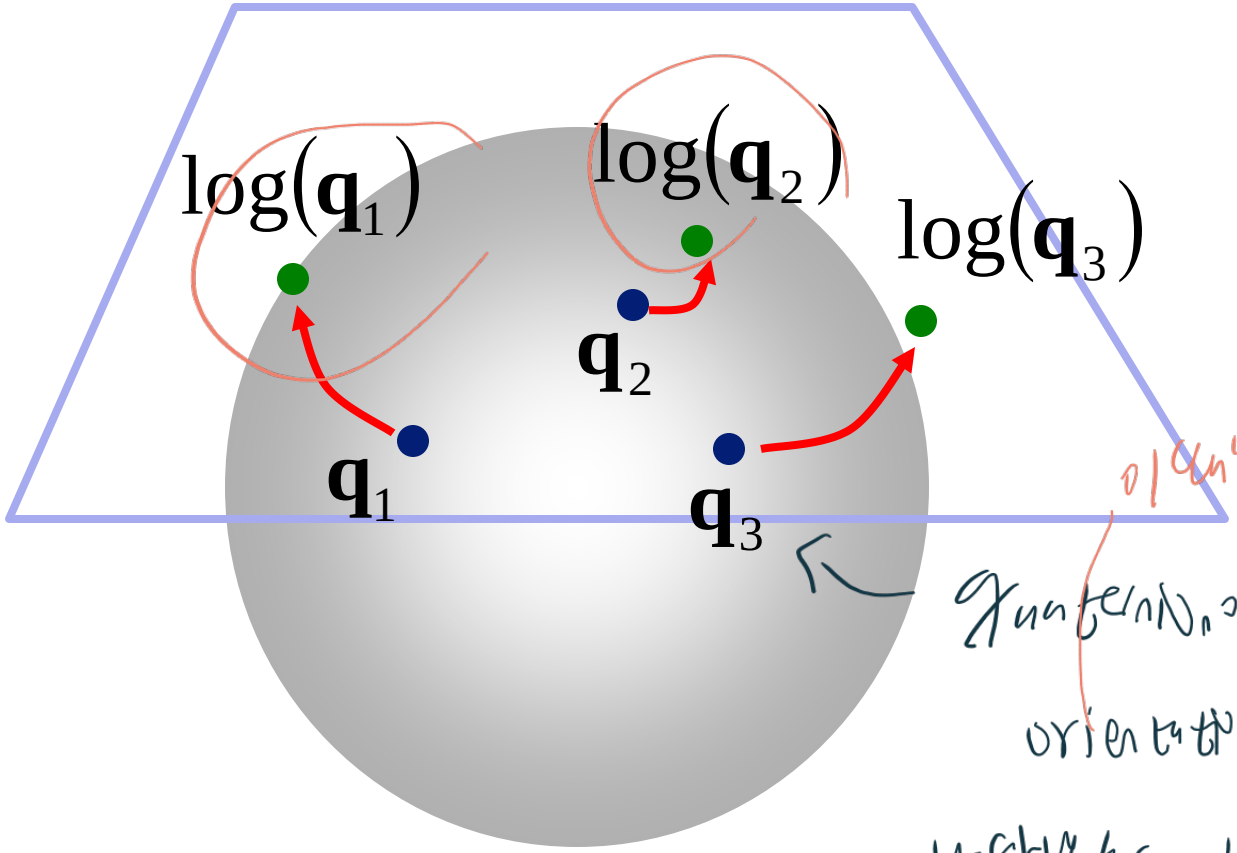
$$\omega(1) = \mathbf{q}^{-1}(1)\dot{\mathbf{q}}(1) = k\log(\mathbf{b}_2^{-1}\mathbf{b}_3)$$

Multi-Linear Method

- Pros
  - Simple, intuitive
  - Inherit good properties of slerp

- Cons
  - Need ordering
    - Eg) De Casteljau algorithm
  - Algebraically complicated

Linearization by Exp/Log



$$\mathbf{q} = \exp(w_0 \log \mathbf{q}_0 + w_1 \log \mathbf{q}_1 + \cdots + w_n \log \mathbf{q}_n)$$

Linearization by Exp/Log

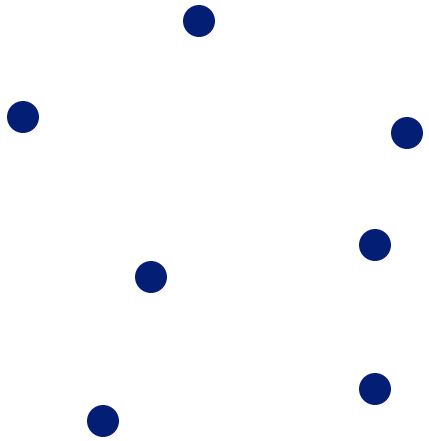
- Pros
  - Simple, easy to implement
  - Versatile

- Cons
  - depends on the choice of the reference frame
  - Singularity near the antipole

Functional Optimization

- We assume that the affine combination is derived from a certain energy function

$$\mathbf{c} = w_1\mathbf{p}_1 + w_2\mathbf{p}_2 + \cdots + w_n\mathbf{p}_n$$



Functional Optimization

- In vector space

Functional

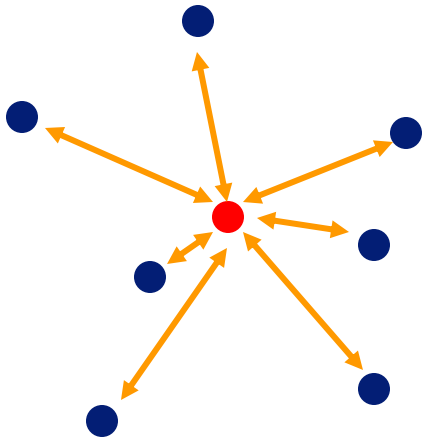
$$f(\bar{\mathbf{p}}) = \frac{1}{2}\sum_i w_i \left\| \bar{\mathbf{p}} - \mathbf{p}_i \right\|^2$$

Minimize

$$\frac{d}{d\bar{\mathbf{p}}} f(\bar{\mathbf{p}}) = \sum_i w_i \left(\bar{\mathbf{p}} - \mathbf{p}_i \right) = 0$$

Weighted sum

$$\bar{\mathbf{p}} = w_1\mathbf{p}_1 + w_2\mathbf{p}_2 + \cdots + w_n\mathbf{p}_n$$



Functional Optimization

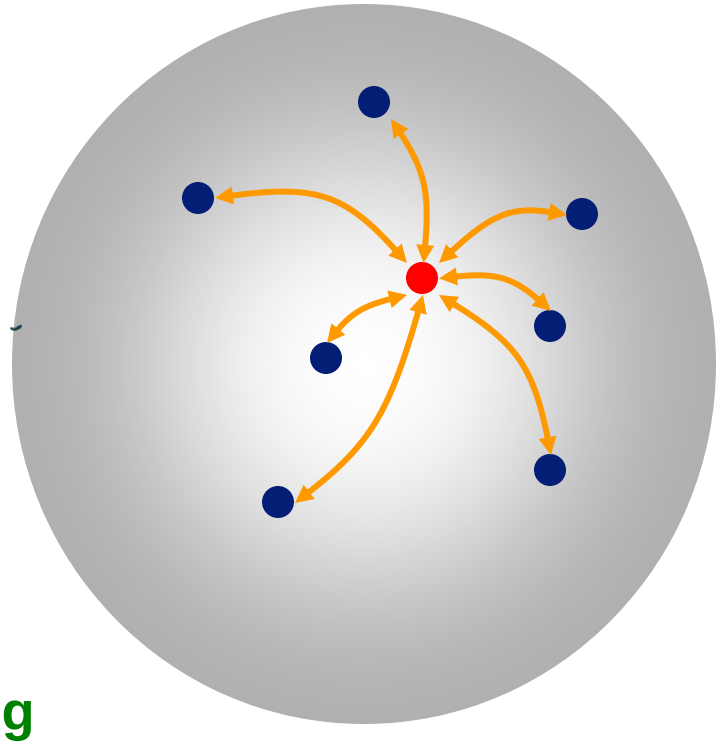
- In unit quaternion space

Functional

$$f(\bar{\mathbf{q}}) = \frac{1}{2}\sum_i w_i \, \mathrm{dist}(\bar{\mathbf{q}}, \mathbf{q}_i)^2$$

Spherical (geodesic) distance

$$\mathrm{dist}(\mathbf{q}_1, \mathbf{q}_2) = \left\| \log(\mathbf{q}_1^{-1}\mathbf{q}_2) \right\|$$



requires a numerical method for root finding

$$\frac{\partial}{\partial \bar{\mathbf{q}}} f(\bar{\mathbf{q}}) = \sum_i w_i \log(\bar{\mathbf{q}}^{-1}\mathbf{q}_i) = (0,0,0)$$

Functional Optimization

- Pros
  - Theoretically rigorous
- Cons
  - Need numerical iterations (New-Rapson)
  - Computationally demanding

Summary

- We don’t have an ultimate solution
- The most suitable solution may be determined by the requirement of the application
- A more specific formulation may lead to a better solution
  - eg) Convolution filters can be viewed as affine combinations of points; however, unlike general affine combinations, they impose a specific ordering on the points.